

Roadmap AutoFi

Posisi Saat Ini

- ✓ Landing page sudah ada
- ✓ Wallet connect sudah bisa
- ✓ Execute sudah trigger popup Phantom
- ✓ Backend masih dummy
- ✓ Smart contract sudah bisa dipanggil

Apa Itu IntentOS?

IntentOS bukanlah sebuah library/dependency, melainkan sebuah konsep arsitektur backend. IntentOS ini adalah sistem yang mengubah keinginan user (intent) menjadi aksi DeFi (execution steps).

Contoh:

User (“maximize profit”) => IntentOS (analisis market, generate strategi, pilih terbaik)

1. **Frontend** - Mambangun Alur Utama (19 April – 22 April / 4 Hari)

a. Buat struktur halaman

```
frontend/app/
├── page.tsx           → Landing
├── connect/page.tsx   → Connect Wallet
├── dashboard/page.tsx → Dashboard
├── strategy/page.tsx  → Select Strategy
├── amount/page.tsx    → Input Amount
├── preview/page.tsx   → Strategy Preview
└── execute/page.tsx   → Execution Screen
```

Hasil yang diharapkan:

Semua halaman tersedia dan bisa diakses tanpa error

b. Implement flow navigasi

```
Landing
↓
Connect Wallet
↓
Dashboard
↓
Select Strategy
↓
Input Amount
↓
Preview
↓
Execute
↓
Dashboard
```

Hasil yang diharapkan:

User bisa mengikuti flow tanpa loncat/stuck

c. **Global state management**

Tersedia data walletAddress, gola, amount, strategy, status

Hasil yang diharapkan:

Data tetap konsisten antar halaman (tidak hilang saat pindah halaman)

d. **Dashboard logic**

Jika belum ada strategi, maka tampilkan tombol pilih strategi. Jika sudah ada strategi, maka tampilkan data portofolio.

2. **Backend - Setup IntentOS (22 April – 23 April / 2 Hari)**

a. **Struktur backend**

```
backend/  
├─ index.js  
├─ services/  
│   ├─ intentEngine.js  
│   ├─ marketData.js  
│   ├─ strategyBuilder.js  
│   └─ riskManager.js
```

Hasil yang diharapkan:

Backend modular dan tidak lagi hardcoded

b. **Endpoint utama**

Endpoint utama pada sistem yaitu POST /intent, dengan input seperti berikut:

```
{  
  "goal": "maximize_profit",  
  "amount": 1000  
}
```

Hasil yang diharapkan:

Frontend bisa request dan menerima response

c. **Implementasi IntentOS**

```
User pilih strategy  
↓  
Ambil template strategy  
↓  
Ambil market data  
↓  
Kirim ke backtest (optimizer)  
↓  
Return optimized strategy
```

Hasil yang diharapkan:

Backend tidak lagi dummy, tapi menghasilkan strategi dinamis

d. **Output IntentOS**

```
{
  "strategy": "aggressive_yield",
  "optimizedSteps": ["swap_SOL_USDC", "lend_USDC"],
  "allocation": [70, 30],
  "estimatedApy": 12.5,
  "risk": "medium",
  "confidence": 0.85
}
```

Hasil yang diharapkan:

Frontend bisa menampilkan preview strategi secara informatif

3. **Backtest (24 April – 26 April / 3 Hari)**

a. **Struktur backtest**

```
backend/backtest/
├─ engine.js
├─ simulator.js
└─ optimizer.js ← WAJIB (baru)
```

b. **Fungsi backtest**

Menguji dan mengoptimasi parameter strategi selama 3 hari

c. **Input backtest**

```
strategy (template)
amount
historical data
parameter variations
```

d. **Proses optimizer**

```
1. Ambil strategi default
2. Generate variasi parameter:
   - allocation %
   - timing
   - threshold
3. Simulasikan semua variasi (3 hari)
4. Hitung:
   - profit
   - drawdown
   - success rate
5. Pilih parameter terbaik
```

e. **Output backtest**

```
{
  "optimizedSteps": ["swap", "lend"],
  "allocation": [70, 30],
  "estimatedApy": 11.8,
  "confidence": 0.87,
  "backtest": {
    "profit": 25,
    "drawdown": 3,
    "successRate": 0.8
  }
}
```

Hasil yang diharapkan:

Strategi menjadi optimal berdasarkan simulasi real, bukan asumsi

4. **IntentOS + Integrasi Backtest (26 April / 1 Hari)**

a. **Flow**

```
User pilih strategy
↓
IntentOS ambil template strategy
↓
IntentOS jalankan backtest (3 hari)
↓
Optimizer mencari parameter terbaik
↓
Return optimized strategy
↓
Frontend tampilkan preview
```

b. **Logic**

```
Strategy: Aggressive Yield

Default:
100% lend

Optimized:
70% lend
30% hold
```

Hasil yang diharapkan:

User tetap memilih strategi, tapi sistem membuatnya lebih optimal

5. **Frontend + Backend - Halaman Preview (27 April / 1 Hari)**

a. **Tampilkan di preview page**

Menampilkan steps, APY, risk, confidence

Hasil yang diharapkan:

User memahami strategi sebelum execute

6. **Blockchain** - Upgrade Smart Contract (27 April - 28 April / 2 Hari)

a. **Update function**

```
execute_intent(goal, amount, steps)
```

b. **Simpan data user**

```
UserPosition:  
- owner  
- amount  
- strategy
```

c. **Validasi**

Hasil yang diharapkan

Smart contract bisa menerima strategi dan menyimpan state user

7. **Frontend** + **Blockchain** - Eksekusi (28 April / 1 Hari)

a. **Flow**

```
Preview  
↓  
Execute  
↓  
Wallet confirm  
↓  
Transaction send
```

b. **Data dikirim**

Data yang dikirimkan berupa goal, amount, steps

Hasil yang diharapkan:

Transaksi berhasil dikirim dan diproses di blockchain

8. **Integrasi DeFi** (29 April / 1 Hari)

a. **Integrasi swap**

Menintegrasikan Jupiter API

b. **Integrasi lending**

Mengintegrasikan Solend/marginfi

c. **Mapping**

Hasil yang diharapkan:

Strategi benar-benar dijalankan di DeFi (bukan hanya log)

9. **Improve Loop** (29 April / 1 Hari)

a. **Simpan data real**

Menyimpan data expected profit dan actual profit

b. **Bandingkan**

Membandingkan selisih performa

c. **Update Confidence**

Jika akurat, maka confidence naik. Tapi jika meleset, confidence menurun

Hasil yang diharapkan:

Strategi semakin akurat dari waktu ke waktu

10. Monitoring + Rebalance (29 April - 30 April / 2 Hari)

a. Monitor market

Jika yield berubah maka harga berubah

b. Trigger ulang IntentOS

c. Rebalance

Hasil yang diharapkan:

Dana user selalu berada di strategi optimal

Ringkasan Alur Sistem

```
User connect wallet
↓
Dashboard
↓
User pilih strategy
↓
Input amount
↓
Backend (IntentOS)
↓
Backtest (3 hari)
↓
Optimize parameter
↓
Preview (optimized)
↓
Execute
↓
Blockchain
↓
DeFi
↓
Profit
↓
Improvement loop
```